Ansible Automation Showcase

Final reference and learning guide for the repository's automation, container, infrastructure-as-code, Kubernetes, GitOps, and CI/CD examples.

Includes verified commands, architecture, security guidance, troubleshooting, and a practical learning path.

FINAL REFERENCE + LEARNING EDITION

Current repository state - August 2026

START HERE

Contents and readiness

Current readiness

Area	Code status	External requirement
Ansible	Inventory, playbooks, role, requirements, and root configuration exist.	Reachable hosts and secure credentials.
Docker	Image builds; health endpoint and clean Gunicorn runtime were verified.	Docker engine for local execution.
Terraform	Provider 3.2.1 lock and current _v1 resources validate without warnings.	A selected Kubernetes context for plan/apply.
AWX	One Kustomize entry point renders the pinned operator and AWX instance.	Kubernetes capacity, storage, ingress/TLS, and production hardening.
Argo CD	Restricted project, application, automated sync, retry, and Kustomize source exist.	Installed Argo CD and one-time bootstrap.
Actions	GitHub-hosted validation works; deployment fails safely when secrets are absent.	Production secrets and Argo CD network reachability.

Source basis

Generated from the repository after commit 604d4aa plus the final README revision. Runtime state in GitHub, AWX, Argo CD, Kubernetes, and managed hosts must still be verified independently.

SYSTEM VIEW

1. Architecture and repository map

The repository combines two automation models. Ansible and AWX run ordered tasks against hosts. Argo CD continuously reconciles Kubernetes desired state from Git. GitHub Actions validates manifests and records deployment evidence.

Step	Component	Responsibility
1	Engineer	Creates a reviewed change to automation or desired state.
2	GitHub Actions	Validates Argo CD and Kubernetes manifests on a hosted runner.
3	Git main	Becomes the auditable source of truth for approved changes.
4	Argo CD	Syncs the exact revision, prunes removed resources, and self-heals drift.
5	Kubernetes	Runs the hello-world workload in a dedicated namespace.
6	Actions evidence	Displays health, operation, history, managed resources, and recent logs.

Repository map

```
ansible-automation-showcase/
|-- .github/workflows/gitops-delivery.yml
|-- ansible.cfg
|-- ansible/                # inventory, playbooks, roles
|-- argocd/                 # AppProject and Application
|-- gitops/hello-world/     # Kubernetes desired state
|-- terraform/              # provider, resources, inputs, outputs
|-- devops-labs/
|   |-- lab1-docker/        # Flask + Gunicorn container
|   `-- lab3-awx/           # operator + AWX instance
`-- output/pdf/             # this guide
```

Repository safety

The repository intentionally has no .gitignore. Always inspect git status before staging. Do not commit .terraform directories, tfstate files, .env files, logs, editor files, Python caches, private keys, tokens, or credentials.

CONFIGURATION MANAGEMENT

2. Ansible and AWX

The inventory defines a RHEL web server and an SSH identity path. Collection requirements install ansible.posix. The root ansible.cfg is discoverable from the project root, including AWX project checkouts.

Core commands

```
ansible-galaxy collection install -r ansible/requirements.yml
ansible-inventory -i ansible/inventory.ini --graph
ansible all -i ansible/inventory.ini -m ansible.builtin.ping
ansible-playbook -i ansible/inventory.ini PLAYBOOK --syntax-check
ansible-playbook -i ansible/inventory.ini PLAYBOOK --check --diff
```

Playbook	Purpose	Review before use
create_users.yml	Creates groups and named user accounts.	Fixed names and UIDs.
disk_monitoring.yml	Calculates disk use, writes a report, and simulates an alert.	Placeholder monitoring URL and ignored alert failures.
healthcheck.yml	Uses the custom app_health module.	Requires appservers inventory and target status path.
patch_scan.yml	Reports available DNF updates.	RHEL-family target only.
setup-nginx.yml	Renders a local Jinja2 example.	Writes under /tmp; no service deployment.
install-nginx-new.yml	Installs and configures NGINX and firewall.	Target OS assumptions and handler behavior.
ansible-nginx-test/site.yml	Demonstrates a reusable NGINX role.	Best base for role testing and reuse.

AWX lab

```
kubectl apply -k devops-labs/lab3-awx
kubectl -n awx get pods
kubectl -n awx get awx awx
```

- The Kustomization pins AWX Operator 2.19.1 and includes the AWX instance.
- The remote operator bundle creates namespace awx; a duplicate Namespace file is unnecessary.
- NodePort is suitable for a local lab. Production needs storage, backups, trusted TLS, ingress, identity, RBAC, and centralized logs.

SMALL SECURE CONTAINER

3. Docker lab

The lab is intentionally one service. The earlier unused Redis container and public Redis port were removed. Flask 3.1.3 runs through Gunicorn 26.0.0 on Python 3.13-slim.

Control	Implementation	Why it matters
Non-root	UID 10001 appuser	Limits the impact of a container compromise.
Read-only root	Compose read_only: true	Prevents unexpected filesystem writes.
Writable temp	tmpfs mounted at /tmp; HOME=/tmp	Supports temporary runtime and Gunicorn control files.
Health	Docker HEALTHCHECK calls /health	Reports whether the application is responsive.
Network	127.0.0.1:8080:8080	Avoids exposing the lab on every host interface.
Process	Two Gunicorn workers with threads	Avoids the Flask development server.

Run and verify

```
docker compose -f devops-labs/lab1-docker/docker-compose.yml up --build -d
curl http://127.0.0.1:8080/health
docker compose -f devops-labs/lab1-docker/docker-compose.yml logs web
docker compose -f devops-labs/lab1-docker/docker-compose.yml down
```

Verified result

The image built successfully. The container reached healthy state, /health returned JSON status healthy, Gunicorn started without errors, and the temporary container and network were removed.

Next production steps

- Pin the Python base image by digest and automate dependency updates.
- Add application tests, vulnerability scanning, SBOM generation, and image signing.
- Set CPU and memory limits in the target runtime.
- Place the service behind a trusted reverse proxy or Kubernetes ingress when remote access is required.

INFRASTRUCTURE AS CODE

4. Terraform lab

Terraform 1.7 or newer uses the HashiCorp Kubernetes provider 3.2.1. The lock file records provider checksums. Current kubernetes_namespace_v1, kubernetes_deployment_v1, and kubernetes_service_v1 resources validate without deprecation warnings.

File	Responsibility
versions.tf	Terraform range and Kubernetes provider constraint ~> 3.2.0.
variables.tf	Kubeconfig path, context, namespace, and replica count with validation.
main.tf	Provider plus namespace, pinned NGINX Deployment, requests/limits, and ClusterIP Service.
outputs.tf	Namespace and a generated kubectl port-forward command.
.terraform.lock.hcl	Resolved provider 3.2.1 and trusted checksums.

Safe workflow

```
terraform -chdir=terraform init
terraform -chdir=terraform fmt -check
terraform -chdir=terraform validate
terraform -chdir=terraform plan

# Optional context override
terraform -chdir=terraform plan   -var='kubeconfig_context=my-context'
```

Resources

- Namespace defaults to portfolio-apps.
- NGINX image is pinned to 1.28.0-alpine instead of latest.
- Replica count defaults to two and must be at least one.
- Container requests 50m CPU/64Mi and limits 250m CPU/128Mi.
- ClusterIP keeps the Service private; use the output port-forward command for local access.

State warning

Terraform state can contain sensitive infrastructure data. This repository has no .gitignore, so inspect git status carefully and store state in a protected remote backend for shared or production use.

DESIRED STATE

5. Kubernetes workload

The Argo CD workload is separate from the Terraform learning workload. Under gitops/hello-world, Kustomize composes a two-replica hello-kubernetes Deployment and a NodePort Service.

Resource	Current configuration	Learning note
Deployment	hello-kubernetes; 2 replicas	Controller maintains desired pod count.
Image	paulbouwer/hello-kubernetes:1.10	Pinned tag, but production should prefer a digest.
Service	NodePort; port 80 to target 8080	Lab exposure; use ingress/gateway and TLS in production.
Kustomization	deployment.yaml + service.yaml	Stable render target for validation and Argo CD.

```
kubectl kustomize gitops/hello-world
kubectl -n hello-world get deploy,pods,svc
kubectl -n hello-world rollout status deploy/hello-kubernetes
kubectl -n hello-world logs deploy/hello-kubernetes --tail=100
```

Production hardening backlog

- Add startup, readiness, and liveness probes.
- Add resource requests/limits and a restrictive security context.
- Use an internal registry and pin the image digest.
- Replace NodePort with ClusterIP behind controlled ingress.
- Add PodDisruptionBudget, topology spread, and NetworkPolicy.
- Add application metrics, alerts, and service-level objectives.

CONTINUOUS RECONCILIATION

6. Argo CD configuration

AppProject portfolio restricts source repository, destination cluster/namespace, and allowed resources. Application hello-world tracks main at gitops/hello-world and deploys into namespace hello-world.

Policy	Value	Effect
Automated prune	true	Removes managed resources deleted from Git.
Self-heal	true	Corrects live drift from desired state.
Allow empty	false	Prevents an empty source from deleting all resources.
CreateNamespace	true	Creates hello-world when permitted.
ApplyOutOfSyncOnly	true	Avoids unnecessary resource writes.
PruneLast	true	Applies desired resources before deleting old ones.
Retry	5 with exponential backoff	Handles temporary reconciliation failures.
History	10 revisions	Keeps bounded revision history.

Bootstrap and operate

```
kubectl apply -f argocd/project.yaml
kubectl apply -f argocd/hello-world-application.yaml
argocd app get hello-world --show-operation
argocd app sync hello-world --prune
argocd app wait hello-world --sync --health --timeout 600
argocd app history hello-world
argocd app logs hello-world --since-seconds 600
```

Prefer a Git revert for rollback because Git remains the audit record. An emergency Argo CD rollback must be followed by a repository correction or automated synchronization may reapply the undesired revision.

VALIDATION AND DEPLOYMENT EVIDENCE

7. GitHub Actions pipeline

Stage	Behavior
Trigger	Pull request, push to main, or manual dispatch for argocd/**, gitops/**, or workflow changes.
Permissions	GITHUB_TOKEN is limited to contents: read.
Concurrency	gitops-production prevents overlapping deployment workflows.
Validation	Pinned Checkout v5 and digest-pinned kubeconform 0.8.0 perform strict validation.
Environment	Deployment references production for secret isolation and optional approval gates.
CLI trust	Argo CD CLI v3.4.2 is downloaded and verified with its published checksum.
Deployment	Syncs GITHUB_SHA, prunes, and waits for operation, sync, and health.
Evidence	Always attempts status, history, resources, and recent pod logs.

Required GitHub configuration

Name	Type	Purpose
production	Environment	Deployment boundary and optional reviewer/branch policy.
ARGOCD_SERVER	Secret	Reachable Argo CD API host.
ARGOCD_AUTH_TOKEN	Secret	Least-privilege automation token for portfolio/hello-world.
ARGOCD_INSECURE	Variable	Lab-only TLS bypass; keep false in production.

Runner topology

- Use GitHub-hosted runners only when Argo CD is reachable through trusted TLS and controlled API access.
- For a private endpoint, use an online ephemeral self-hosted runner with private network access.
- If CI-to-cluster access is prohibited, keep CI validation only and let Argo CD pull automatically.

Observed pipeline state

Manifest validation has passed on a GitHub-hosted runner. Deployment stopped at the explicit configuration check because ARGOCD_SERVER was absent. This is the intended safe behavior until the production environment is configured.

BUILD THE LAB

8. End-to-end setup runbook

1. Clone and inspect

```
git clone https://github.com/mderangula/ansible-automation-showcase.git
cd ansible-automation-showcase
git status --short
```

2. Validate each component

```
ansible-galaxy collection install -r ansible/requirements.yml
ansible-playbook -i ansible/inventory.ini   ansible/playbooks/test-connection.yml --syntax-check
docker compose -f devops-labs/lab1-docker/docker-compose.yml config --quiet
terraform -chdir=terraform init
terraform -chdir=terraform validate
kubectl kustomize gitops/hello-world >/dev/null
kubectl kustomize argocd >/dev/null
kubectl kustomize devops-labs/lab3-awx >/dev/null
```

3. Install and bootstrap

- Install Argo CD in namespace argocd using the platform-approved method.
- Apply argocd/project.yaml before argocd/hello-world-application.yaml.
- Optionally apply the AWX Kustomization in a lab cluster.
- Create GitHub environment production with server and token secrets.

4. Prove delivery

- Open a pull request modifying gitops/hello-world and verify validation.
- Merge after review and watch the Argo CD sync job.
- Confirm the exact commit is Synced and Healthy.
- Review status, history, managed resources, logs, and Kubernetes events.

5. Prove recovery

- Introduce a controlled failure in a lab branch.
- Observe validation or health failure and identify its evidence.
- Revert the Git change, resynchronize, and verify recovery.
- Record time to detect and time to restore.

RUN RESPONSIBLY

9. Security and operations

Security checklist

- Use protected main, required pull requests, CODEOWNERS, and successful validation checks.
- Keep production secrets out of pull-request jobs and use environment-scoped secrets.
- Use dedicated automation identities with minimum permissions and regular rotation.
- Pin actions, images, providers, and downloaded tools; review updates intentionally.
- Use trusted TLS. Do not use ARGOCD_INSECURE outside disposable labs.
- Store Terraform state in a protected remote backend for team environments.
- Back up AWX and Argo CD data and test restore procedures.
- Forward platform and application logs to centralized observability.

Operational cadence

Frequency	Checks
Per change	Diff, validation, exact revision, sync, health, logs, and smoke test.
Weekly	Failed workflows, drift, orphan warnings, restarts, pending updates, disk/capacity.
Monthly	Credential review/rotation, dependency updates, backup restore check, RBAC review.
Quarterly	Rollback exercise, disaster recovery, token revocation, and platform upgrade rehearsal.

Promotion model

For multiple environments, use separate directories or repositories, distinct Argo CD Applications and AppProjects, environment-specific GitHub protection, and promotion through reviewed commits. Avoid one shared credential with unrestricted access to every cluster.

DIAGNOSE AND PRACTICE

10. Troubleshooting and learning path

Symptom	Likely cause	First checks
Workflow queued	No matching runner or concurrency lock.	Runner status/labels and active gitops-production run.
Validation fails	YAML/schema error.	kubeconform file path, API version, indentation, and resource fields.
Missing ARGOCD_SERVER	Environment/secret absent.	GitHub production environment and secret names.
Argo CD timeout	Network, DNS, TLS, or firewall.	Runner connectivity and certificate trust.
App not found	Bootstrap missing or wrong Argo CD context.	kubectl -n argocd get applications and CLI server.
Permission denied	AppProject or token RBAC restriction.	Source, destination, resource allowlists, and token role.
Degraded	Workload not healthy.	describe, events, scheduling, image pulls, probes, and logs.
Ansible unreachable	SSH/network/user/key/Python issue.	Direct SSH and ansible ping with -vvvv.

Learning sequence

Stage	Exercise	Proof
1. Ansible	Inventory graph, ping, syntax, check mode, then role twice.	Second role run is idempotent.
2. Docker	Build, inspect user, test health, review logs.	Healthy non-root container with clean logs.
3. Terraform	Init, validate, plan with two contexts.	Explain inputs, state, lock file, and proposed resources.
4. Kubernetes	Render and apply workload in a lab.	Two Ready pods and reachable service.
5. Argo CD	Bootstrap app and create deliberate drift.	Drift is detected and self-healed.
6. CI/CD	Submit valid and invalid pull requests.	Interpret validation and deployment evidence.
7. Recovery	Break image, observe failure, revert Git.	Restore Synced and Healthy state.

Quick reference

```
git status --short
ansible-playbook -i ansible/inventory.ini PLAYBOOK --check --diff
docker compose -f devops-labs/lab1-docker/docker-compose.yml up --build
terraform -chdir=terraform validate
kubectl kustomize gitops/hello-world
argocd app get hello-world --show-operation
argocd app wait hello-world --sync --health --timeout 600
gh run list --workflow gitops-delivery.yml
gh run view RUN_ID --log-failed
```

Final success criteria

- Code and generated manifests validate.
- Docker lab is healthy and logs are clean.
- Terraform plan targets the intended context and excludes secrets/state from Git.
- Argo CD application is Synced and Healthy at the approved commit.
- Deployment evidence is visible and actionable.
- Rollback, backup, and recovery have been exercised.